Self-Hosted LLMs for Accessible Software Engineering: A Drift-, Bias- and Cost-Aware Evaluation on an 8 GB Consumer GPU

Anonymous Author(s)
Affiliation withheld for review

***Abstract*—Large language models (LLMs) now write and repair user-interface code, yet comparisons of LLMs rarely measure accessibility, usually test one-shot prompts, sometimes let a model judge its own output, and seldom report cost. We present an evaluation framework that measures accessible software engineering as it happens in practice. Six capability dimensions anchored in WCAG 2.2 are scored by real-browser oracles (axe-core and 27 keyboard and semantics probes). The *Dynamic Contextual Drift Index* (DCDI) measures how much accessibility adherence an agent loses over a long, tool-using session, against a fresh-context control and a length-matched padded control. *Cross-Model Blind Peer Evaluation* (CMBPE) is an anonymised, family-recused, order-permuted and length-adjusted jury for alt text, calibrated against ratings by blind and low-vision people. *Capability-per-Dollar-Second* (CPDS) relates capability to cost and latency with bootstrap Pareto membership. We apply the framework to four open-weight models that a small organisation can self-host for free on one 8 GB consumer GPU (Qwen3 8B, Qwen2.5-VL 7B, Gemma 3 4B and Ministral 3 8B, 4-bit), with 268 logged model calls on 12 November 2026. Every reported value is computed by committed code from logged responses; code, prompts, logs and scores are public.**

***Index Terms*—accessibility, WCAG 2.2, large language models, LLM-as-a-judge, agents, evaluation, alt text**

I. Introduction

Most web pages still fail automated accessibility checks, and code assistants now write a growing share of user-interface code. Whether that code is accessible therefore matters for blind, low-vision, motor-impaired and keyboard-only users. Studies of LLMs and accessibility report that generated code often misses accessibility requirements unless they are requested, and that models can fix many violations once they are pointed out [1], [2]. These studies usually test one or two models with one-shot prompts, judge outputs with a single (sometimes the same) model, and do not report what an evaluation or a remediation costs.

Four practical questions remain open. Does a model build accessible interfaces *by default*, or only when told to? Does it keep honouring accessibility while it works through a long, tool-using session, where its context fills with history [3], [4]? Can LLM judges rate accessibility content such as alt text reliably, given their known position, length and self-preference biases [5], [6], [7]? And what does AI-assisted remediation cost per page, in money and in time?

We answer these questions with a framework built around automated, browser-based oracles and three new measures (DCDI, CMBPE and CPDS), and we apply it to the setting most relevant to organisations without an API budget: small open-weight models served locally on one 8 GB consumer GPU. Our research questions are:

- **RQ1** How do the models perform across six accessibility-engineering capability dimensions (C1–C6)?
- **RQ2** Do LLM agents keep honouring accessibility requirements as a long, tool-using session grows?
- **RQ3** Does a blind, family-recused, order-permuted, length-adjusted jury (CMBPE) judge accessibility content more reliably than single LLM judges?
- **RQ4** Which models are Pareto-efficient in capability, cost and latency, and what does AI-assisted remediation cost?

Contributions. (1) A capability taxonomy that maps general LLM abilities to six WCAG 2.2-anchored tasks with automated oracles, including 27 keyboard and semantics probes, each validated on a passing and a failing fixture. (2) DCDI, a drift measure with a counterfactual fresh-context control and a padded control that separates the effect of context length from the effect of the session's history. (3) CMBPE, a bias-controlled jury with a Bradley–Terry model of position and length bias. (4) CPDS, a scenario-weighted efficiency score with bootstrap Pareto membership. (5) A fully logged study of four self-hosted models, with code, prompts, raw scores and call logs released.

Scope. This study evaluates 4–8B open-weight models at 4-bit quantisation, served by llama.cpp [8] on one consumer GPU. It does not evaluate frontier closed models. The harness supports them, but running them needs paid API access. Claims about the frontier, or about closed versus open models, are therefore out of scope here.

II. Related Work

LLMs and accessible code. Aljedaani et al. studied the accessibility of web code generated by ChatGPT and its ability to repair violations [1]. CodeA11y adds accessibility-aware suggestions and reminders to an AI coding assistant and evaluates them with developers [2]. We complement such studies

with repeated, multi-model measurement, a repair corpus with known faults, long agentic sessions and cost.

Automated accessibility testing. Rule engines such as axe-core [9] check a subset of WCAG success criteria [10] in a rendered page; keyboard operability, focus management and widget state largely escape them. We pair axe-core with browser-driven probes (Playwright [11]) and treat all automated results as lower bounds on failures.

LLM-as-a-judge. LLM judges favour the first-shown or longer answer and their own outputs [5], [6], [7]. CMBPE combines several known mitigations (both orders, anonymisation, recusal) with an explicit Bradley–Terry model [12] of position and length bias, and calibrates the jury against human ratings.

Long contexts and agents. Models use information in long contexts unevenly [3], and effective context is often shorter than the advertised window [4]. Agentic coding benchmarks measure task success [13] but not whether non-functional requirements stated once at the start are still honoured many steps later. DCDI measures exactly that.

Alt-text evaluation. Kreiss et al. showed that context matters for the usefulness of image descriptions to blind and low-vision (BLV) readers and collected BLV ratings [14]. ContextRef provides crowd ratings of descriptions in Wikipedia context [15]. We use both as calibration data for LLM judges.

III. Framework

A. Oracles and the rule-level pass rate

Pages are rendered in headless Chromium 156.0.8078.4 at 1280×800 and served from a local HTTP server that blocks every other host. axe-core 4.14.0 runs with the WCAG 2.x Level A and AA rule tags. Pages with timers are checked again after 6 s, and a rule fails if it failed in either run. 27 probes cover what axe cannot see. Three generic probes run on every page: tab reachability, visible focus and no keyboard trap. The others cover widget patterns: form errors associated with their fields and not shown by colour alone, dialog focus handling, tabs, accordions, disclosure navigation, comboboxes, sliders, switches, tooltips, live regions, data-table headers, sort controls, pause controls for moving content, multi-step focus management, captions, landmarks, headings, language of parts, contrast after a theme toggle and accessible names that are not placeholder-only. Every probe passes its accessible fixture and fails its inaccessible fixture. Probes find widgets by visible text anchors, never by implementation details, and accept any keyboard path that satisfies the success criterion.

For an artefact x, let $Q(x)$ be the applicable checks: every axe rule that evaluated at least one node in scope, plus every applicable probe. The pass rate is

$$a(x) = |\{q \in Q(x) : q \text{ passes}\}| \,/\, |Q(x)|. \tag{1}$$

Task-specific functional checks (for example "an empty submit is blocked") confirm that the page does what was asked.

TABLE I
Capability dimensions, experiments and scores.

Dimension	Experiment	Score
C1 conformance auditing	E1-Audit	S_1 = micro-F1 on 192 known faults
C2 accessible UI generation and repair	E1-Gen, E1-Repair	$S_2 = (S_{\text{gen}} + S_{\text{rep}})/2$
C3 agentic execution of a multi-page build	E2	S_3 = mean in-session adherence
C4 text alternatives for images	E3c	S_4 = CMBPE expected win rate
C5 adherence as context grows	E2	$S_5 = 1 - \text{clip}(\text{DCDI}, 0, 1)$
C6 conflicting requests and honest claims	E2, E1-Gen	S_6 = (pressure score + (1−overclaim rate))/2

B. Capability taxonomy

Table I lists the six dimensions and their scores $S_k \in [0, 1]$. The composite capability score is the weighted geometric mean $\text{CCS}_m = \exp\left(\sum_k w_k \ln \max(S_{m,k}, 0.01)\right)$ with $w_k = 1/6$, reported only for models with all six scores; a geometric mean keeps a weak dimension from being masked by strong ones.

C. Dynamic Contextual Drift Index (DCDI)

An agent works through a fixed sequence of steps on one workspace. After each step t of run r we score the in-session adherence $a_{r,t}$ on the step's scope and checks. A *fresh control* replays the same step in a new conversation on a copy of the workspace snapshot $S_{r,t-1}$, with the same system prompt and tool budget, giving $a^0_{r,t}$. Over the foundation and regular steps T_{reg} that are functional in both arms,

$$\text{DCDI}_m = \frac{\sum_{r,t \in T_{\text{reg}}} (a^0_{r,t} - a_{r,t})}{\sum_{r,t \in T_{\text{reg}}} a^0_{r,t}}. \tag{2}$$

DCDI is 0 without drift, 1 when all adherence is lost and negative when history helps. A *padded control* at checkpoints K replays the step after irrelevant history whose length matches the session's prompt length $\Lambda_{r,t}$ within $\pm 10\,\%$. It splits the drift at the checkpoints into a length and a history component, $\text{DCDI}_K = [\sum(a^0 - a^{\text{pad}}) + \sum(a^{\text{pad}} - a)]/\sum a^0 = \text{DCDI}_{\text{len}} + \text{DCDI}_{\text{hist}}$. The drift rate β_m is the OLS slope of $a^0 - a$ on $\log_2(\Lambda/8192)$ with step fixed effects (adherence lost per doubling of context).

D. Cross-Model Blind Peer Evaluation (CMBPE)

Judges from different model families compare two anonymised candidates for the same image in both orders. Candidate texts are stripped of quotes, markdown and lead-ins such as "Image of", then given random opaque ids. A verdict is dropped from aggregation when the judge's family equals either candidate's family (recusal). With $y = 1$ if the first-shown candidate wins,

$$P(y = 1) = \sigma(\theta_a - \theta_b + \delta_j + \gamma_j \, \Delta\ell), \tag{3}$$

where $\Delta\ell$ is the standardised length difference, δ_j and γ_j are judge j's position and length biases, and $\sum \theta = 0$. The model

is fitted by penalised maximum likelihood (ridge 10^{-3} on θ) with image-cluster bootstrap intervals. A generator's score is its expected win rate $S_4(m) = \mathrm{mean}_{g \neq m}\, \sigma(\theta_m - \theta_g)$. Self-preference η_j is estimated from judge j's own-family verdicts with θ fixed at the panel estimate. Position consistency PC_j is the share of pairs with the same verdict in both orders. Baselines are a single judge (one seeded order, no recusal or adjustment), a naive majority panel and a length-only scorer.

E. Capability-per-Dollar-Second (CPDS)

For model m and task family f (nudged generation, repair), with capability score CS, mean billed cost per task C and median latency L,

$$\mathrm{CPDS}^{(\alpha,\beta)} = \frac{CS}{\phi(C/c_0)^{\alpha}\,\phi(L/l_0)^{\beta}}, \quad \phi(z) = 1+\log_{10}(1+z), \tag{4}$$

with $c_0 = \$0.01$ and $l_0 = 1\,\mathrm{s}$. The scenarios are batch remediation ($\alpha = 1, \beta = 0$, on repair), interactive assistant ($\alpha = 0.5, \beta = 1$, on generation) and balanced ($\alpha = \beta = 1$). A model's Pareto-membership probability π_m is the share of B bootstrap resamples in which no other model is at least as capable, cheap and fast and strictly better in one of them. Self-hosted models have $C = 0$ (electricity is not metered), so CPDS reduces to a latency-adjusted capability on the stated hardware.

IV. Study Design

A. Models, hardware and serving

Table III lists the models. Qwen3 8B [16] is a text model with thinking on by default; it runs E1 and E2. Qwen2.5-VL 7B [17], Gemma 3 4B [18] and Ministral 3 8B [19] are vision-language models from three families and form the E3 generator pool and jury; Qwen2.5-VL 7B also runs E1. All weights are the publishers' Q4_K_M GGUF files, pinned by SHA-256. They are served by llama.cpp b12004 [8] through its Vulkan backend on the open Mesa NVK driver, on an NVIDIA RTX 2060 SUPER (8 GB), an Intel i7-9700K and 32 GB RAM (Ubuntu 26.04). Every layer is on the GPU and one model is resident at a time. Qwen3 8B uses its native 32,768-token window with an 8-bit KV cache and may emit up to 32,768 tokens, as its model card recommends. The other models may emit up to 8,192. The harness sends no sampling parameters. The server applies each model's Ollama-library defaults (Qwen3 8B: temperature 0.6, top-k 20, top-p 0.95; Qwen2.5-VL 7B: 0.0001; Gemma 3 4B: 1.0, 64, 0.95; Ministral 3 8B: 0.15), so Qwen2.5-VL 7B is close to greedy. Images are encoded with at most 1,280 visual tokens (Gemma 3 4B uses its fixed 256). A prompt longer than the window is rejected by the server and recorded as context exhaustion, never truncated. Requests are serialised, so latencies are those of one request on this machine. Table II summarises the serving speed and GPU memory measured during the study (`experiments/local/extra_values.py`). The desktop session shares the GPU, so the memory figures include about 1.3 GB that the study does not use.

TABLE II
MEASURED SERVING SPEED AND GPU MEMORY (MEDIAN OVER STUDY CALLS; VRAM OVER 10 S SAMPLES WHILE THE MODEL WAS WORKING; ALL PROCESSES).

Model	Decode tok/s	Prompt tok/s	VRAM MiB
Qwen3 8B	25.7	311	?
Qwen2.5-VL 7B	32.1	312	6853
Gemma 3 4B	?	?	?
Ministral 3 8B	?	?	?

TABLE III
MODELS ACTUALLY RUN (GENERATED FROM `CONFIG/MODELS.YAML`). V: VISION. TTC: TEST-TIME COMPUTE. USE: 1 = E1, 2 = E2, G = E3 GENERATOR, J = E3 JUDGE. SELF-HOSTED MODELS HAVE NO PER-TOKEN PRICE.

Model	Open	Arch.	V	TTC	$ in/out	Use
Qwen2.5-VL 7B (local)	✓	vision-language 7B	✓	–	0/0	1

Open-weight prices are those of the hosting provider used.

B. E1: generation, repair and audit (C1, C2, C6)

Generation. 20 interface tasks: form with validation, modal dialog, tabs, accordion, disclosure navigation, data table, image gallery with lightbox, auto-rotating carousel, autocomplete combobox, switch, slider, status message, article structure, video, pagination, tooltip, file upload, multi-step form, consent banner and icon links. Each runs *unguided* and *nudged* (one added sentence: "Make sure it is accessible and conforms to WCAG 2.2 Level AA."), with 3 samples each. The model must return one self-contained HTML document; a missing or truncated block scores 0. A second turn asks whether the page conforms to WCAG 2.2 AA (JSON). S_{gen} is the mean over nudged samples of $a(x) \cdot \mathbb{1}[\text{functional}]$. The *default gap* is $S_{\text{gen}}(\text{nudged}) - S_{\text{gen}}(\text{unguided})$. The *overclaim rate* is the share of functional, imperfect pages ($a(x) < 1$) that the model claims conform; it is a lower bound, because automated checks miss failures.

Repair. 12 accessible base pages: 4 adapted W3C "Before and After" demonstration pages [20] and 8 authored pages. 12 mutation operators inject faults: image alt removed (SC 1.1.1), label replaced by a placeholder, contrast reduced, icon-button name removed, icon-link name removed, page language removed, button turned into a `div`, heading demoted, positive `tabindex`, focus indicator removed, invalid ARIA attribute and page title removed. A seeded search yields 48 faulty pages with 4 faults each (192 faults, 16 per operator), and every injected fault is detected by the oracles. A repair scores $s = \max(0, (\text{resolved} - \text{regressions})/4) \cdot \mathbb{1}[\text{preserved}]$, where preservation requires visible-text similarity ≥ 0.95, every base-page id and DOM tag-sequence similarity ≥ 0.85. S_{rep} is the mean page score.

Audit. The model lists the problems of each faulty page as JSON. Reports are matched one-to-one to injected faults by success criterion and element. S_{audit} is the micro-F1 over the

192 faults; unmatched reports count as false positives, a lower bound on precision.

C. E2: long agentic sessions (C3, C5, C6)

The model builds a small clinic website with three file tools (`list_files`, `read_file`, `write_file`) in a sandboxed workspace. The system prompt states the WCAG 2.2 AA requirement once; no user message mentions accessibility. There are 24 steps: 4 foundation pages (services, appointments, contact, news), then 14 regular features (hero banner, site search, accordion, form validation, confirmation dialog, cookie banner, gallery, language switcher, pagination, back-to-top button, social links, dark-mode switch, news ticker, pain-level slider) and 6 *pressure* requests that conflict with accessibility (placeholder-only labels, light-grey footer text, a click-only tile, removing focus outlines, colour-only validation errors, disabling zoom), in a fixed random order. A step ends when the model answers without a tool call or after 10 tool calls; history is never truncated or summarised. Pressure steps score 1 if the model keeps the page accessible, 0.5 for a partial outcome and 0 if it complies in an inaccessible way. Fresh controls run at every step, padded controls at positions 8, 12, 16, 20 and 24 (only where the session reaches them), and an end-of-session audit re-scores every earlier step.

D. E3: alt text and judges (C4)

D_{BLV}: 68 descriptions in article context, each rated by four BLV participants [14]. D_{CR}: 204 descriptions in Wikipedia context with crowd ratings [15]. Images are the datasets' own stimuli (222 distinct images, all retrieved). The judges, one per family, rate every item on a 5-point scale (pointwise) and compare pairs in both orders. *E3a* pairs two D_{BLV} descriptions of the same image whose BLV ratings differ by at least 0.5 (23 pairs). *E3b* pairs each original description with four degraded versions (another item's first sentence appended, repeated text, another image's description, the first three words), giving 1,058 known-answer pairs. *E3c* has each generator write alt text for 120 seeded D_{CR} images. With three generators all three pairs per image are judged, and after recusal each pair keeps exactly one judge, from the third family.

E. Statistics

We report 95 % percentile bootstrap intervals ($B = 2000$, seed 20271113) that resample the natural cluster: task (E1-Gen), page (E1-Repair, E1-Audit), run $\times$ step (E2) and image (E3) [21]. All models share the same resamples, so differences are paired. We call two models different only when the interval of their difference excludes 0. Hypothesis tests are one-sided bootstrap tests with Holm correction [22]. Inter-judge agreement is Krippendorff's α [23].

F. Pre-registered hypotheses and their status in this study

The framework's pre-registered hypotheses are: H1, larger siblings drift less than smaller ones (vendor pairs); H2, the best closed model's C4 score exceeds the best open model's; H3, at least one open-weight model has $\pi \geq 0.5$ on the batch-remediation frontier; H4, (a) some single judge prefers its own family and (b) CMBPE beats the median single judge on known-answer pairs; H5, for most models nudged generation beats unguided generation. With four local open-weight models, H1 and H2 cannot be tested and H3 holds trivially. H4 and H5 are tested as planned.

G. Changes from the original plan

The original plan targeted frontier and hosted models through paid APIs. Without an API budget, this study runs the plan's documented local profile, with these changes (all logged, dated and motivated in `experiments/DEVIATIONS.md` before the affected calls): llama.cpp instead of Ollama, because the latter has no GPU backend on the open driver; a third local judge family (Ministral 3 8B), without which recusal removes every E3c verdict; a 32,768-token output budget for Qwen3 8B (the plan's rule: the model's maximum output); the image-token cap; the E2 padding transcript written by Qwen3 8B instead of Claude Haiku 5.5, kept to the turns that fit its window; and one E2 run (seed 101) instead of three. No metric, prompt, task or scoring rule was changed. No public pre-registration tag was created before data collection, so this study is not pre-registered. The run's code was published (commit `7544be2`) before all but a handful of smoke-test calls, which used the same code.

V. Results

The result paragraphs are written from the generated values once every run has finished; until then the tables and figures below show the data available so far, and red markers show values that have not been computed yet.

VI. Threats to Validity

Oracle coverage. Automated checks find only some WCAG failures, so $a(x)$, the overclaim rate and audit precision are bounds, not exact values. The probes were validated on fixtures, not on every possible implementation.

Model scale and quantisation. The models are 4–8B parameters at 4-bit quantisation. Larger or unquantised models, and other conversions of the same weights, may behave differently. Results describe these exact files (SHA-256 in the repository).

Context window. Qwen3 8B's 32,768-token window bounds E2: a session ends when its history no longer fits. DCDI is then measured over the steps completed, and late padded checkpoints may not exist.

Sampling and repetitions. Provider defaults make Qwen2.5-VL 7B near-greedy, so its three E1 samples are close to identical. E2 has one run, so drift intervals reflect steps within one session order.

Jury size. With three families, each E3c pair is judged by one non-recused judge, and local vision models write and judge alt text less well than larger models. Agreement with BLV ratings is measured on 68 descriptions.

TABLE IV

CAPABILITY PROFILE, DRIFT AND EFFICIENCY (GENERATED). S_1–S_6: DIMENSIONS OF TABLE I; CCS ONLY WITH ALL SIX SCORES; CPDS-BATCH ON REPAIR, CPDS-INTER ON NUDGED GENERATION; π: PARETO-MEMBERSHIP PROBABILITY ON THE BATCH FRONTIER. "–": NOT MEASURED FOR THIS MODEL.

Model	S_1	S_2	S_3	S_4	S_5	S_6	CCS	DCDI	$\text{CPDS}_{\text{batch}}$	$\text{CPDS}_{\text{inter}}$	π_{batch}
Qwen2.5-VL 7B (local)	–	–	–	–	–	–	–	–	–	0.00	–

TABLE V

JUDGE RELIABILITY (E3, GENERATED). r: PEARSON CORRELATION OF POINTWISE SCORES WITH THE MEAN HUMAN RATING (BLV READERS; CROWD WORKERS); ACC: ACCURACY ON KNOWN-ANSWER PAIRS; PC: POSITION CONSISTENCY; γ: LENGTH BIAS; η: SELF-PREFERENCE (SINGLE JUDGES).

Method	r_{BLV}	r_{crowd}	$\text{Acc}_{\text{known}}$	PC	γ	η
Qwen2.5-VL 7B (local) (single)	–	–	–	–	–	–
Gemma 3 4B (local) (single)	–	–	–	–	–	–
Ministral 3 8B (local) (single)	–	–	–	–	–	–
Naive panel	–	–	–	–	–	–
CMBPE	–	–	–	–	–	–
Length only (baseline)	–	–	–	–	–	–

Hardware. Latency, and hence CPDS, depends on the GPU, the driver (NVK) and the quantisation; the cost of electricity is not included.

Process. The plan was frozen in the repository but not publicly time-stamped before data collection; all later changes are listed with reasons.

VII. CONCLUSION

We presented a framework that measures accessible software engineering with browser-based oracles, a drift index with counterfactual controls, a bias-controlled jury and a cost-aware efficiency score, and applied it to four models that run for free on one consumer GPU. The harness, prompts, tasks, corpus, call logs and scores are public, so the same study can be repeated on other hardware or extended to API models by adding keys.

ARTIFACTS AND REPRODUCIBILITY

Code, configuration and data are on the `local-run-2026-11` branch of the project repository. `experiments/local/serve_llamacpp.sh` serves each model with the exact flags used, `experiments/local/run_local.sh` runs the whole study, and `experiments/runs/` holds every call record (`calls.jsonl`), score file and E2 transcript. All tables, figures and values in this paper are regenerated by `python -m analysis.run_all -paper experiments/results/local-run-2026-11`. The study logged 268 calls at a total token cost of $0.

REFERENCES

[1] W. Aljedaani *et al.*, "Does ChatGPT generate accessible code? Investigating accessibility challenges in LLM-generated source code," in *Proceedings of the 21st International Web for All Conference (W4A)*, 2024.

[2] P. Mowar *et al.*, "CodeA11y: Making AI coding assistants useful for accessible web development," in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2025, arXiv:2502.10884.

[3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[4] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg, "RULER: What's the real context size of your long-context language models?" in *Conference on Language Modeling (COLM)*, 2024.

[5] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[6] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, L. Kong, Q. Liu, T. Liu, and Z. Sui, "Large language models are not fair evaluators," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[7] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[8] ggml-org, "llama.cpp: LLM inference in C/C++," https://github.com/ggml-org/llama.cpp, 2026, release b12004 used.

[9] Deque Systems, "axe-core: Accessibility engine for automated web UI testing," https://github.com/dequelabs/axe-core, 2026, version 4.14.0 used.

[10] World Wide Web Consortium, "Web content accessibility guidelines (WCAG) 2.2," W3C Recommendation, https://www.w3.org/TR/WCAG22/, 2023.

[11] Microsoft, "Playwright," https://playwright.dev, 2026.

[12] R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: I. the method of paired comparisons," *Biometrika*, vol. 39, no. 3/4, pp. 324–345, 1952.

[13] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024.

[14] E. Kreiss, C. Bennett, S. Hooshmand, E. Zelikman, M. Ringel Morris, and C. Potts, "Context matters for image descriptions for accessibility: Challenges for referenceless evaluation metrics," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.

[15] E. Kreiss, E. Zelikman, C. Potts, and N. Haber, "ContextRef: Evaluating referenceless metrics for image description generation," in *International Conference on Learning Representations (ICLR)*, 2024.

[16] Qwen Team, "Qwen3 technical report," arXiv:2505.09388, 2025.

[17] ——, "Qwen2.5-VL technical report," arXiv:2502.13923, 2025.

[18] Gemma Team, "Gemma 3 technical report," arXiv:2503.19786, 2025.

[19] Mistral AI, "Ministral 3 8B Instruct (2512)," https://huggingface.co/mistralai/Ministral-3-8B-Instruct-2512-GGUF, 2025.

[20] W3C Web Accessibility Initiative, "Before and after demonstration (BAD)," https://www.w3.org/WAI/demos/bad/.

[21] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[22] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[23] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 2nd ed. Sage, 2004.